

A data fusion framework based on heterogeneous information network embedding for trigger-action programming in IoT

Gang Wu, Liang Hu, Xuelin Mao, Yongheng Xing, Feng Wang*

College of Computer Science and Technology and the Engineering Research Center of Network Technology and Application Software, Ministry of Education, Jilin University, Changchun 130012, China

ARTICLE INFO

Keywords:

Internet of Things
Trigger-action programming
Heterogeneous information network embedding
Artificial Intelligence of Things

ABSTRACT

Trigger-action programming (TAP) in the Internet of Things (IoT) enables users to easily customize the desired behaviors of connected entities, such as smart devices and online services, by creating trigger-action rules, also known as recipes. An example of such a rule is “If Sunset, Then Turn on lights”. As potential trigger-action combinations grow exponentially, there has been significant interest in automatically generating recipes based on users’ natural language instead of relying on manual creation. In this article, we present TAPFuser, an IoT data fusion framework designed to encode natural language and recipes, aiming to map human demand and recipes using a heterogeneous information network (HIN) embedding approach. Firstly, we divide TAP data into machine and human levels based on their sources. We then model these two levels of data as an IoT-HIN and employ HIN embedding techniques to learn vector representations of nodes, which is beneficial for recipe recommendation. To enhance the adaptive capabilities of TAPFuser, we propose a specific metagraph-guided random walk method that captures the level-aware heterogeneity of the IoT-HIN. Finally, we formulate the mapping between natural language generated by the human level and recipes generated by the machine level as a multi-label classification problem, where each node represents a natural language description, and the label corresponds to the category of the entity included in the recipe. We conducted multi-label classification experiments using the IFTTT dataset, and the results demonstrate the effectiveness of our proposed framework.

1. Introduction

The advent of emerging communication technologies, such as 5G and IPv6, dramatically promotes the development of IoT. It has also led to considerable growth in the number of connected devices. It is predicted that the number of IoT devices will reach 76 billion by 2025.¹ Nowadays, connected entities, whether smart devices or online services are ubiquitous and able to interact with each other, with humans, and with the environment (Corno, De Russis, & Monge Roffarello, 2021).

Trigger-action programming (TAP) (Mi, Qian, Zhang, & Wang, 2017), as an increasingly popular Internet of Things (IoT) paradigm, enables users to manually create TAP rules (AKA recipes) in the form of “if something triggers, then perform an action”, providing significant convenience in people’s daily lives. Based on this paradigm, a limited IoT ecosystem connects numerous entities, such as smart devices or online services. Fig. 1 illustrates the potential combinations of functions provided by different interconnected entities, which facilitate the provision of diverse services based on current scenarios. In subfigure

(b) of Fig. 1, functions (i) and ③ combine to create an interaction link for cooling the room when the temperature is high, while functions (iii) and ② merge to dehumidify the space in the presence of high humidity. Additionally, users are free to customize and select specific combinations of functions to meet their individual demands.

However, as the number of entities and their functions increases, the potential combinations of triggers and actions grow exponentially. For instance, in an IFTTT scenario (Mi et al., 2017), when the online service *Weather Underground* detects a sunset, the smart device *Phillips Hue* turns on the lights. In our current experimental dataset, this scenario includes 14 triggers and 10 actions, resulting in a minimum of 140 possible recipes (Yu, Hua, & Julien, 2021). With the IoT ecosystem expanding infinitely, it becomes increasingly time-consuming and frustrating for users to browse and create recipes manually.

In response to this problem, researchers have shown interest in the automatic generation of recipes based on users’ natural language, rather than relying on manual creation (Corno et al., 2021; Yao, Li, Gao,

* Correspondence to: No. 2699 Qianjin Street, Changchun City, China.

E-mail addresses: wugang17@mails.jlu.edu.cn (G. Wu), hul@jlu.edu.cn (L. Hu), maoxl18@mails.jlu.edu.cn (X. Mao), xingyh18@mails.jlu.edu.cn (Y. Xing), wangfeng12@mails.jlu.edu.cn, feng_w@jlu.edu.cn (F. Wang).

¹ <https://www.statista.com/statistics/471264/iot-number-of-connecteddevices-worldwide/>

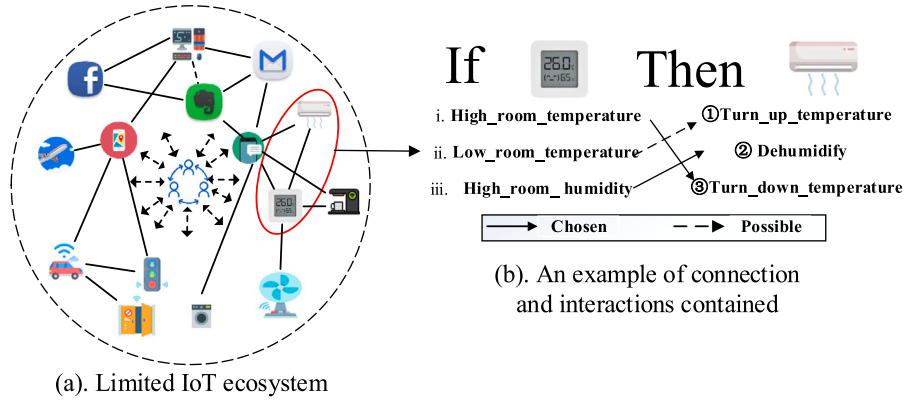


Fig. 1. The development of IoT ecosystem and the division of levels. (a) Connections among entities in a limited IoT ecosystem. (b) The connection of the smart panel and air conditioner. There are at least $3 \times 3 = 9$ possible combinations of functions as interaction links. One can choose the combination, such as functions (i) and ③, (iii) and ②, according to their demands.

Sadler, & Sun, 2019; Yusuf, Jamal, Jiang, & Lo, 2022a; Yusuf, Jiang, & Lo, 2022b). These methods involve parsing the user's natural language description into the functions of entities within recipes. However, the training process of these methods primarily focuses on leveraging the natural language descriptions provided by users. As a result, the model's ability to capture the intricate semantics inherent in entities and their functions remains limited.

To address the aforementioned deficiencies, we propose TAPFuser, an IoT data fusion framework that encodes natural language and recipes. The goal is to establish a mapping between human demands and recipes using a heterogeneous information network (HIN) embedding technique. In TAPFuser, we divide TAP data into the machine and human levels based on their sources. In an expanding IoT ecosystem, many interaction links are automatically formed at the machine level, representing combinations of entities' functions known as interaction contexts. Simultaneously, these links are recommended to fulfill potential human demands at the human level, typically described in natural language as demand descriptions. Then, an IoT-HIN (Hu, Gong, Xing, & Wang, 2019a) is established to integrate the interaction contexts and human demands. An edge is established for interaction contexts if a composition relationship exists between entities' functions. We utilize a natural language processing (NLP) tool to extract meaningful words and phrases, treating them as nodes, and establish edges between these text nodes and word/phrase nodes. These nodes are converted into low-dimensional vector representations via the HIN embedding techniques. The learned vector representations contain structural information between nodes and can be employed in various downstream IoT applications, such as IoT security (Zangeneh & McCabe, 2020), IoT service recommendation (Kim, An, Lee, & Lee, 2017; Kim, Jung, Ji, Hwang, & Rho, 2019), and IoT automation (Wei et al., 2021).

To improve the adaptive ability of the TAPFuser, we propose a specific metagraph-guided random walk (SMRW) method, which sets a hyperparameter to control the probability of walking at different levels. After HIN embedding procedure, the mapping problem is formulated as a multi-label classification task to determine the category of the entity for each recipe's natural language description. In summary, the contributions of this article are as follows:

1. We propose an IoT data fusion framework, namely TAPFuser, to encode natural language and recipes, which aims to establish a mapping between human demand and recipe.
2. We propose to divide TAP data into the machine and human levels based on their source, which serves as the basis for mapping. We model information from both machine and human levels as an IoT-HIN and convert it into low-dimensional vector representations using HIN embedding technique. To improve the adaptive ability of TAPFuser, we customize a specific metagraph-guided random walk method to capture the level-aware heterogeneity of the IoT-HIN.

3. We perform multi-label classification experiments on the IFTTT dataset to test our proposed framework. The results show that our proposed framework can encode human demands and outperform other compared methods.

The rest of this article is organized as follows. Section 2 reviews the related works. Preliminaries are described in Section 3. We give the technical details of the designed framework in Section 4. Section 5 represents our experiments. Furthermore, Section 6 concludes the article.

2. Related work

In this section, we first introduce graph-based data representation and HIN embedding in IoT. Then, we briefly review the practices of the TAP pattern.

2.1. Graph-based data representation and embedding in IoT

The utilization of graphs in data representation and processing offers increased flexibility and convenience. In IoT, Hu et al. (2019a) designs a heterogeneous graph-based IoT-HIN to describe the flow of information transmission among IoT devices. They employ an enhanced matrix factorization technique to cluster IoT devices based on semantic similarity. Besides, the graph is also applied to other applications in IoT, such as the clustering of Point of Interest (Hu, Xing, Gong, Zhao, & Wang, 2019c) and mobile crowd sensing (Wang, Hu, Sun, Hu, & Zhao, 2018).

Furthermore, for solving the problems of existing graph analysis methods, which suffer high computation and space when faced with massive data, the HIN embedding technique provides a striking pattern (Cai, Zheng, & Chang, 2018).

Applying the network embedding to IoT, the article (Hu, Tang, Zhang, & Wang, 2020) builds a vehicular ad hoc network based on the co-occurrence phenomenon by modeling vehicle trajectory records into a graph. The network embedding technique is used to learn the vector representation of drivers in the network, and they can recommend potential friends for drivers based on the similarity of the vectors.

Considering its ability to preserve and process data, we use graph structure to model and integrate IoT data. Applying HIN embedding methods, we can obtain latent representations of data effectively.

Table 1
Arguments of the two recipes.

Component	Trigger	Action	Recipe
	Channel.function	Channel.function	Description
Recipe of Fig. 2(a)	Coffee machine.cook coffee	Delivery robot.delivery	Give me my coffee if the coffee is ready
Recipe of Fig. 2(b)	Smart panel.temperature in room	Air conditioner.turn down indoor temperature	Turn down the temperature if it is above 30 °C in room

2.2. TAP pattern in IoT

TAP has become popular recently because of its user-friendliness and automation. Some interfaces and platforms, such as IFTTT² and Zapier,³ are available as off-the-shelf products for the TAP diagram (Corno, De Russis, & Monge Roffarello, 2019). As emerging communication technologies such as 5G rise, the research of TAP has attracted considerable attention from around the world (Ur, McManus, Pak Yong Ho, & Littman, 2014; Ur et al., 2016; Yu et al., 2021).

Corno et al. (2019) considers entity interaction contexts as rules and builds a rules knowledge graph through the RDF individual relationship and the OWL class relationship with rules. Then RecRule, a recommendation system, is proposed to recommend recipes to end-users based on a semantic reasoning process on this knowledge graph. Further, they develop a conversational agent, called HeyTAP, that considers users' intentions and preferences and the connected entities to recommend recipes (Corno et al., 2021). Hu, Wu, Xing, and Wang (2019b) focuses on human demand descriptions at the human level as context-aware data in IFTTT. They construct an IoT context graph to represent the different semantic relationships among these context-aware data. Then, the network embedding technique is used to generate latent representations for interactions. Recent studies (Kim, Suh, & Lee, 2022; Xing, Hu, Zhang, Wu, & Wang, 2021) utilize network embedding to learn the representation of triggers and actions from interaction contexts at the machine level. The learned representations are used in AIoT applications, such as link prediction and classification of triggers and actions.

However, the approaches mentioned above study either the demand descriptions from the human level or the interaction contexts data from the machine level, which did not differentiate the IoT from a human-machine perspective, resulting in inefficient utilization of IoT data. Our proposed framework integrates the information from human and machine levels by graph modeling and then fuses the representations of recipes through HIN embedding.

3. Preliminaries

In this section, we first introduce the key concepts required for the TAP. Then, we give the related definition of the TAP data fusion framework.

3.1. Preliminaries for TAP

TAP, a trigger-action rule providing automatic execution service for web applications and IoT devices, can realize the automation of connected entities in an event-driven form. Instead of writing code in a complicated programming language, TAP enables users who may not have the programming ability to create recipes for their daily needs. There are some websites for TAP, including IFTTT and Zapier, and millions of personal recipes have been created. In this article, we focus on the data from IFTTT.com. Fig. 2 illustrates two recipes in IFTTT with their descriptions.

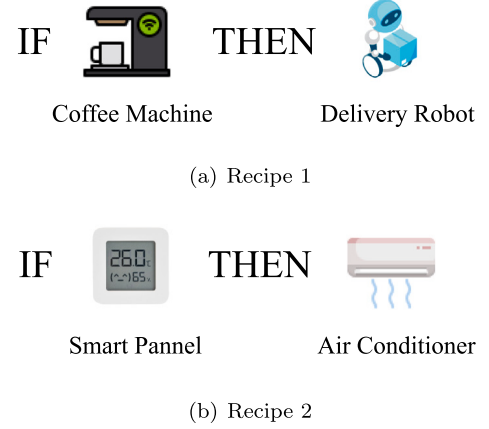


Fig. 2. Trigger-action rules. (a): Recipe of “Give me my coffee if it is ready”.; (b): Recipe of “Turn down the temperature if it is above 30 °C in the room”.

The structure and components of recipes are presented in Table 1. Each recipe on IFTTT consists of the *trigger channel*, the *trigger function*, the *action channel*, and the *action function* in the form “if something triggers, then perform an action”, where the trigger function is event or condition, abbreviated as trigger, and the action function is instruction or operation that can be executed, referred to as action. The *channel* is a smart device or service provider. For example, the recipe of Fig. 2(a) has a trigger channel coffee machine and an action channel delivery robot, which provide cooking coffee and delivery functions, respectively. There are diverse channels, including connected devices, web applications, and other service providers, which provide various functions as events for triggers and actions. These channels also belong to different service categories.⁴ For example, Instagram belongs to social networks, and air conditioners belong to household appliances.

Meanwhile, IFTTT allows users to share their recipes publicly, meaning users can prefer and use recipes others have already created according to individual needs. Moreover, users can edit natural language descriptions for recipes to express their intention.

3.2. Related definition

This section introduces the definitions of relevant components in our proposed framework.

Definition 1 (Machine Level In TAP). The machine level, denoted as ML , is defined as $ML = \{Ent, Fnc, Lin\}$, where:

- $Ent = \{Ent_1, Ent_2, \dots, Ent_n\}$ is the set of entities, including IoT devices and online services.
- $Fnc = \{Fnc_1, Fnc_2, \dots, Fnc_n\}$ is the set of functions offered by Ent .
- $Lin = \{Lin_1, Lin_2, \dots, Lin_n\}$ is a link set generated by the combination of functions in Fnc_i . For example, Lin_i represents the combination of Fnc_i and Fnc_j .

² <https://ifttt.com>

³ <https://zapier.com>

⁴ <https://ifttt.com/services>

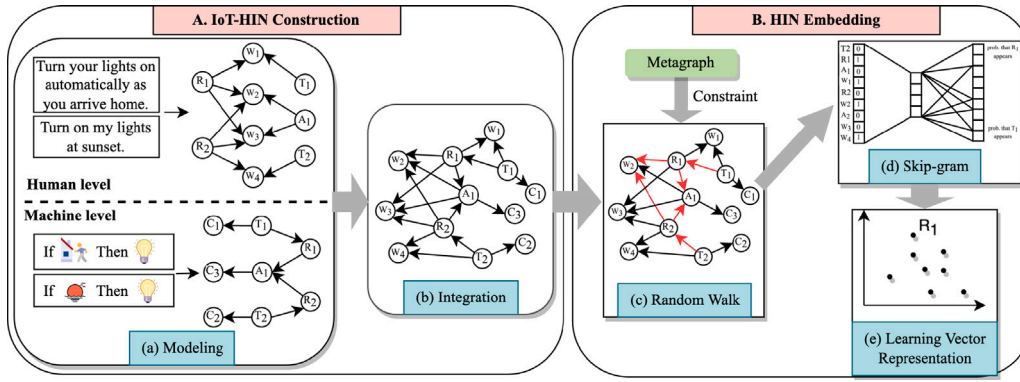


Fig. 3. Overview of the proposed framework. A. IoT-HIN Construction: (a). Utilize the graph to model information from human and machine levels as a network. (b). Integrate the two modeled networks as IoT-HIN. B. HIN Embedding: (c). Random walk from each node in IoT-HIN to capture its context. (d)&(e). Introduce the skip-gram model to incorporate the heterogeneous contexts in random walk paths, then learn each node's vector representation in IoT-HIN.

Definition 2 (Human Level In TAP). The human level is represented as a set of attribute information $HL = \{A_1, A_2, \dots, A_n\}$ provided by users, where $A_i = \{a_{i1}, a_{i2}, \dots, a_{in}\}$ can be the set of natural languages generated by users.

Large volumes and heterogeneity characterize TAP data from human and machine levels. These TAP data consist of multiple heterogeneous entities (e.g., air conditioner, robot, and coffee machine) and links (e.g., combine or edit). Connecting these devices and links is a HIN about IoT.

Definition 3 (IoT Heterogeneous Information Network). The IoT-HIN is denoted as a directed graph $G = (V, E, T)$ in which each node and each link is associated with their mapping function $\phi: V \rightarrow T_V$ and $\varphi: E \rightarrow T_E$ respectively, where $|T_V| + |T_E| > 2$. T_V and T_E denote the sets of node and link types in IoT-HIN.

In IoT-HIN, multiple types of nodes and edges exist, encompassing single and multiple relationships between two nodes. In this article, we adopt the concept of metapaths to model the single relationship between nodes, which is defined as follows.

Definition 4 (Metapath on IoT-HIN). A metapath scheme P is defined as a sequence in the form of $T_{v_1} \xrightarrow{T_{e_1}} T_{v_2} \xrightarrow{T_{e_2}} \dots \xrightarrow{T_{e_n}} T_{v_n}$, wherein $T_e = T_{e_1} \circ T_{e_2} \circ \dots \circ T_{e_n}$ defines the composite link type sets between node types T_{v_1} and T_{v_n} , where n is the path length we pre-defined.

While metapaths can effectively represent the strict single-path semantics between two nodes (Dong, Chawla, & Swami, 2017), they may not adequately capture the multiple semantics present in the IoT-HIN (Zhang, Fang, Liu, Wu, & Zhang, 2020). Consequently, an alternative method is needed to preserve relationships while fully maintaining the neighborhood structure of nodes in IoT-HIN. This method should be capable of accommodating the diverse semantics between nodes within the network.

Definition 5 (Metagraph on IoT-HIN). A metagraph M is a directed acyclic graph (V_M, E_M, T_M) defined on IoT-HIN, where V_M and E_M is the set of nodes and links in metagraph M , respectively. T_M denotes the set of node and edge types in metagraph M . In general, a metagraph has only a single source node u with 0 in-degree and a single target node type v with 0 out-degree, which is defined as $M = (V_M, E_M, T_{V_M}, T_{E_M}, u, v)$, where $V_M \subseteq V, E_M \subseteq E$ constrained by $T_{V_M} \subseteq T_V$ and $T_{E_M} \subseteq T_E$, respectively.

Indeed, metagraphs offer a powerful approach for jointly modeling multiple relationships between nodes in HINs (Zhang et al., 2020; Zhang, Yin, Zhu, & Zhang, 2018). In the context of IoT-HIN, where the semantics between nodes are diverse, we leverage metagraphs to

effectively capture and represent the complex structure of the IoT-HIN, which includes multiple relationships between pairs of nodes. This approach allows us to comprehensively analyze and utilize the rich relationships in IoT-HIN for various tasks and applications.

4. The proposed framework

This section introduces TAPFuser in detail, including IoT-HIN construction and HIN embedding. First, we divide TAP data into human and machine levels based on their source. Then, an IoT-HIN is constructed to integrate information from two levels. Finally, a specific metagraph-guided random walk (SMRW) based HIN embedding method is proposed to learn the vector representations for the IoT-HIN. The mapping between human demands and recipes is formulated as a multi-label classification problem on IoT-HIN. The overall schematic illustration of the framework is shown in Fig. 3.

4.1. The hierarchical division of TAP data

In this section, we focus on TAP data within the context of IFTTT. Fig. 4 shows that the IoT ecosystem comprises diverse smart devices and online services. These entities are heterogeneous and possess predefined functions. IFTTT allows users to combine the functions of different entities, enabling the creation of recipes that represent interaction contexts at the machine level. Additionally, users can edit natural language descriptions for the recipes they create. These descriptions contain rich semantic information that reflects users' demands, making recipes representative of demand descriptions at the human level.

The hierarchical division of TAP data is motivated by two factors. Firstly, data from different sources exhibit heterogeneity. At the human level, demand descriptions exhibit high semantic complexity due to their dynamic and random nature. In contrast, interaction contexts at the machine level are relatively static, as the functions and interfaces of IoT devices are predetermined during manufacturing. Secondly, the hierarchical division of TAP data effectively distinguishes two spaces with differing semantic complexity, facilitating the mapping between human demands and recipes. This article aims to automate the generation of recipes based on the user's natural language, eliminating the need for manual creation. To achieve this goal, we propose an IoT data fusion framework.

4.2. IoT-HIN construction

The graph has powerful and flexible representation ability. This article uses the graph to model the heterogeneous interaction contexts and demand descriptions. The key idea is to use a unified graph to represent both the interaction contexts and the demand descriptions, enabling the learning of representations for both levels of information.

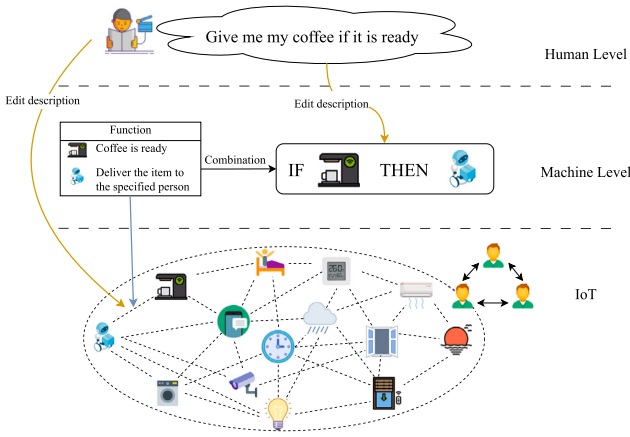


Fig. 4. The generation of data occurs at two levels in IFTTT. Users pick up the interconnected devices they need on the IFTTT platform and specify the execution order in the form of “If *trigger* Then *action*.” Then, they edit text descriptions for their recipes based on their needs.

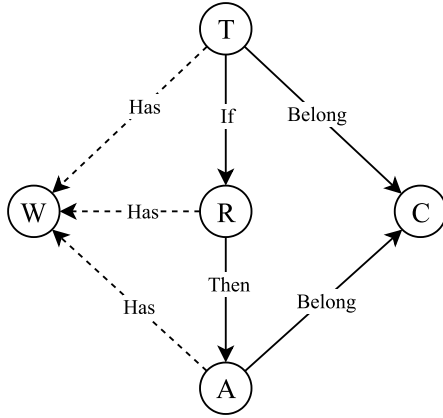


Fig. 5. Illustration of IoT-HIN. The dotted black edges indicate the integration processes of the human demand descriptions and the entity interaction contexts.

To achieve that, we first abstract the recipes, triggers, actions, channels, and descriptions as objects for the TAP with interaction contexts and demand descriptions. Then, the interaction relationships among connected entities and descriptions are extracted to build an IoT-HIN. Its definition can be found in Section 3.2. Taking the IFTTT dataset as an example, we can present the structure schematic of IoT-HIN in Fig. 5 with node types including recipe (R), trigger (T), action (A), channel (C), word (W), and links indicate the *If* (T-R), *Then* (R-A), *Belong* (T-C, A-C), *Has* (R-W, T-W, A-W) relationship types.

4.2.1. Model interaction contexts from the machine level

For interaction contexts from the machine level, an entity interaction contexts network (ICN) in IoT is constructed, shown in Fig. 6. First, trigger and action are treated as nodes with different types. And then, the recipes, serving as a bridge for connecting the machine and human levels, are viewed as nodes in the network. The channels providing triggers and actions are also taken as nodes. Finally, through their context relationships (*If*, *Then*) and affiliation relationships (*Belong*), these nodes are linked. In this article, the ICN is defined as $\mathbb{G}^I = (\mathbb{V}^I, \mathbb{E}^I)$.

4.2.2. Model demand descriptions from the human level

The demand description, being in natural language text form, exhibits characteristics of randomness and dynamics. To effectively extract users' demands from this text, this article employs Natural Language Processing (NLP) techniques. These techniques aim to identify

and extract important sentence components from the demand descriptions, to reduce the high semantic complexity inherent in natural language. The part-of-speech tagger, a function of NLTK,⁵ is first used to extract meaningful words, including verbs, nouns, verb phrases, and noun phrases from the descriptions of trigger, action, and recipe. These words express the semantic information of the demand descriptions and are viewed as nodes, which are linked using the contain relationship (*Has*). This way, we finish converting unstructured text descriptions to a human demand description network (HDDN). The HDDN is denoted as $\mathbb{G}^H = (\mathbb{V}^H, \mathbb{E}^H)$ and Fig. 7 shows an instance of HDDN.

4.2.3. Construct IoT-HIN

This article constructs two networks, ICN and HDDN, modeled by interaction contexts and demand descriptions from the machine and human levels. As aforementioned, each trigger, action, and recipe has its corresponding description. The two networks inherit this unique characteristic, which we apply in our integration process. In detail, we remove the nodes R_d , T_d , and A_d with text descriptions in HDDN and let the corresponding nodes R , T , and A in ICN connect their word nodes with edges of type *Has*. The dotted black edges in Fig. 5 indicate the edges created. After integrating, demand descriptions and interaction contexts are represented as an IoT-HIN.

4.3. HIN embedding in IoT-HIN

Inspired by metapath2vec (Dong et al., 2017), we apply HIN embedding to encode the information in IoT-HIN, which can convert nodes into low-dimensional vector representations. Given an IoT-HIN $\mathbb{G} = (\mathbb{V}, \mathbb{E}, \mathbb{T})$, IoT-HIN embedding aims to learn a mapping function $f : v \in \mathbb{V} \rightarrow \mathbb{R}^{|\mathbb{V}| \times d}$, where $\mathbb{R}^{|\mathbb{V}| \times d}$ means d dimensional vector representation of nodes.

In IoT-HIN, heterogeneous relationships between nodes exist, requiring a method to represent these relationships effectively. Inspired by Pathsim (Sun, Han, Yan, Yu, & Wu, 2011), the metapath is used to represent the relationships between different types of nodes. For example, in Figs. 8(a) and 8(b), the metapaths *RTR* and *RAR* describe a composite relation between two recipes and present that two recipes have the same trigger and action, respectively. In addition, there are multiple different relationships among pair of nodes, such as *If*, *Then*, and *Has*, which are connected via multiple paths. For example, in Fig. 8(d), both *If* and *Then* relationships exist between the two recipes. In that scenario, we use the metagraph (Zhang et al., 2018) to model the structure of IoT-HIN with multiple relationships.

4.3.1. Metagraph-guided random walk

To generate meaningful node sequences in IoT-HIN, designing an effective random walk strategy that encompasses the complete semantics is crucial. One approach uses metapath-constrained random walk, which can effectively model nodes with a single relationship (Dong et al., 2017). However, for more complex heterogeneous structures and richer semantics in IoT-HIN, the metagraph-guided random walk is preferred as it integrates multiple metapaths (He et al., 2019). By leveraging the metagraph, the integrated structure of IoT-HIN from ICN and HDDN can be represented. Nevertheless, traditional metagraph-guided random walk methods (Zhang et al., 2018) often assume equal probabilities for walking across different types of nodes. In IoT-HIN, where machine and human layers have distinct semantics, the importance of node types may differ. Consequently, the uniform metagraph-guided random walk method falls short in capturing the complete structural information of IoT-HIN.

To solve this problem, we propose SMRW, a specific metagraph-guided random walk strategy to capture the level-aware heterogeneity

⁵ <http://www.nltk.org/>

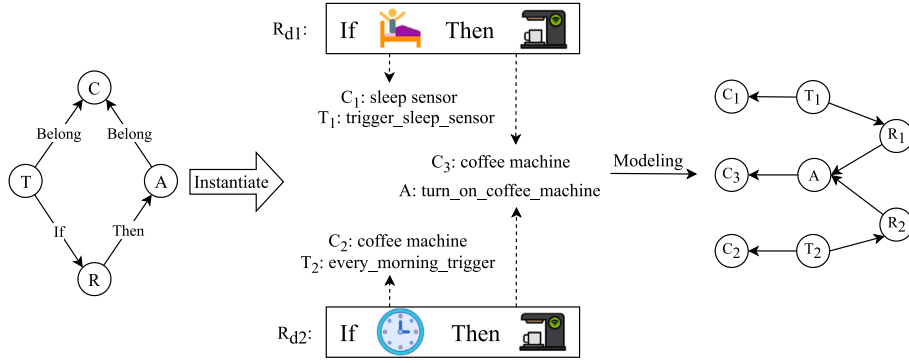


Fig. 6. Illustration of ICN in IoT and an instance. Nodes R, T, A, and C denote recipes, triggers, actions, and channels. R_i , T_i , A_i and C_i are their instances of two examples.

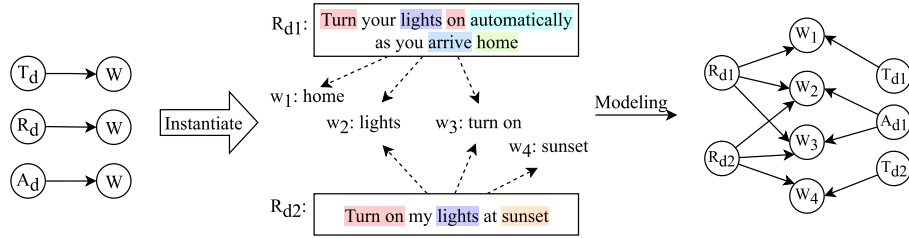


Fig. 7. Illustration of HDDN and an instance. Node types R_d , T_d , and A_d represent the descriptions of recipes, triggers, and actions. Node w means meaningful words or phrases in descriptions. R_{di} , T_{di} , A_{di} represent the instances of nodes R_d , T_d and A_d . W_i means the instances of nodes w .

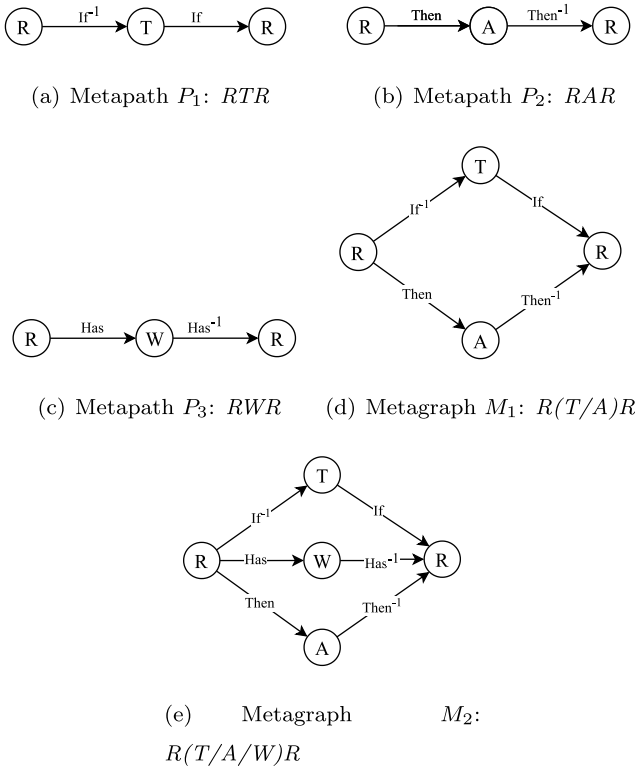


Fig. 8. The metapaths and metagraphs in IoT-HIN. The P_i and M_i are the markups of metapath and metagraph, respectively.

of the IoT-HIN. Given an IoT-HIN $\mathbb{G} = (\mathbb{V}, \mathbb{E}, \mathbb{T})$ and a metagraph M , the transition probability $p(v^{i+1}|v^i, M)$ at step i is defined as follows:

$$p = \begin{cases} \lambda \cdot \frac{1}{|N_t(v^i)|} & (v^i, v^{i+1}) \in \mathbb{E}^I, \phi(v^i) \neq \phi(v^{i+1}) \\ (1 - \lambda) \cdot \frac{1}{|N_t(v^i)|} & (v^i, v^{i+1}) \in \mathbb{E}^H, \phi(v^i) \neq \phi(v^{i+1}) \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

where ϕ is the node type mapping function, N_t is the t type of neighborhood of v^i . λ is a hyperparameter that adjusts the probability of walking the ICN and HDDN. Every step of the random walk will follow the pattern of the metagraph we defined in advance and will be stopped until it reaches the pre-defined length.

Compared to the traditional method (Zhang et al., 2018), which regards the probability of walking nodes from different levels as uniform, SMRW designs a specific random walk strategy to obtain the importance of different levels. The larger λ presents higher priority on the machine level than the human level.

4.3.2. Learn vector representations

The sequence sets generated by SMRW can preserve the semantic and structural relationships between different types of nodes. For learning vector representations of nodes, we introduce the word-context concept (Mikolov, Chen, Corrado, & Dean, 2013a) in a text corpus and map a random walk sequence and node to a sentence and word in the corpus, respectively. Then, skip-gram is applied on walk sequences to maximize the occurrence probability of the context $C(v)$, given a node v . The objective function is:

$$\arg \max_{\theta} \sum_{v \in \mathbb{V}} \sum_{t \in \mathbb{T}} \sum_{u \in C_t(v)} \log p(u|\psi(v)) \quad (2)$$

where $C_t(v)$ denotes the neighborhood of node v with type t and $\psi(\cdot)$ is a mapping function for embedding: $\psi(v) : v \in \mathbb{V} \rightarrow \mathbb{R}^{|\mathbb{V}| \times d}$.

To solve the problem of the quite expensive cost of calculating the objective function, we introduce a negative sampling technique for efficient optimization in Mikolov, Sutskever, Chen, Corrado, and Dean

(2013b). Given a negative sample size K , Eq. (2) is updated as follows:

$$\log \sigma(\psi(u) \cdot \psi(v)) + \sum_{k=1}^K E_{u'_k \sim P_{neg}(v)} \log \sigma(-\psi(u'_k) \cdot \psi(v)) \quad (3)$$

where $\sigma(x) = 1/(1 + \exp(-x))$ and $P_{neg}(v)$ is a pre-defined noise distribution that generates negative samples.

Then we use a stochastic gradient descent (SGD) algorithm to optimize the objective Eq. (3):

$$\psi(v) := \psi(v) + \eta \frac{\partial \log p(u|\psi(v))}{\partial \psi(v)} \quad (4)$$

The whole algorithm framework is shown in Algorithm 1. After the above steps, we can obtain a low-dimensional real-valued vector for each node in the IoT-HIN. These nodes can be used to measure the similarity among nodes in low-dimensional space. The closer the distance between node vectors, the more similar the semantic information contained in the nodes.

Algorithm 1 SMRW-based HIN embedding

Input: IoT-HIN $G = (V, E, T)$, a specific metagraph M , walks per node h , walk length g , embedding dimension d , neighborhood size c .

Output: The latent node embedding.

```

1: initialize  $walks = \{\}$ ;
2: for iteration  $x = 1$  to  $h$  do
3:   for all nodes  $v$  in  $V$  do
4:      $walk[1] = v$ ;
5:     for iteration  $y = 1$  to  $g$  do
6:       extract  $u$  by utilizing Eq. (1);
7:        $walk[y+1] = u$ ;
8:     end for
9:     Append  $walk$  to  $walks$ ;
10:  end for
11: end for
12:  $\psi = \text{SGD}(walks, c, d)$  according to Eq. (3)–(4);
13: return  $\psi$ 
```

4.4. Mapping between human demands and recipes

The goal of establishing a mapping between human demands and recipes is to facilitate the generation of recipes based on users' natural language descriptions, ultimately promoting IoT automation. To achieve this goal, the mapping process between human demands and recipes is formulated as a multi-label classification problem. As described in Sections 3.1 and 4.1, human demands are expressed through natural language descriptions, while recipes consist of various components, including triggers, actions, channels, and categories. During the embedding process, triggers, actions, and channels are utilized. Therefore, each recipe's category is chosen as the label in the mapping process. Specifically, each recipe is assigned two labels: one for the trigger channel and another for the action channel.

Given the recipes' demand descriptions and interaction contexts, we first generate vector representations for each node, where the representations of description nodes R are used as input for multi-label classification tasks. The partially labeled nodes are then used to train a classifier. The idea behind this is that similar natural language descriptions will likely have similar components. We aim to capture the inherent similarities and patterns within the demand descriptions by generating vector representations for the description nodes. This information is then leveraged in the multi-label classification process to predict the components of the recipes.

5. Experiments

To demonstrate the rationality and effectiveness of our proposed framework, we conducted multi-label classification experiments on the

Table 2

The details of selected recipes.

Category	Recipe
Voice assistants	547
Environment control and monitoring	409
Mobile devices and accessories	280
Location	315
Weather	323
Security and monitoring systems	453
Calendars and scheduling	271
Smart hubs and systems	467
Appliances	183
DIY electronics	102
Power monitoring and management	113
Lighting	809

Table 3

The statistics of IoT-HIN.

Node	Edge	Class
# trigger: 202	# R-T: 2173	# category: 12
# action: 460	# R-A: 2173	
# word: 2656	# R-W: 10594	
# recipe: 2173	# T-W: 1572	
	# A-W: 2818	

IFTTT dataset. We evaluate the classification performance and compare it with other methods to illustrate the superiority of TAPFuser. We analyze the impact of different parameters on TAPFuser and discuss the insights found from the experimental results. All experiments are performed in the Python 3.10 environment with 3.70 GHz Intel(R) Core(TM) i9-10900K CPU and 64 GB RAM.

5.1. Experimental setup

5.1.1. Dataset

The IFTTT (Yu et al., 2021) is an IF-THEN rule execution service platform that covers hundreds of services, from IoT devices to web applications. The raw dataset was obtained by Yu et al. (2021) with the IFTTT platform web scrape as of August 2021. It contains 50,067 recipes, of which 2648 are related to IoT applications. For reducing noises, we filter the raw dataset based on the popularity and quantity of recipes and their relevance to IoT applications. First, we select the recipes related to IoT applications and select the category based on the number of these recipes. Categories with recipes less than 100 are removed. And then, we extract the channels, triggers, actions, and recipes that belong to these remaining categories. After that filtering process, 12 categories and 2173 recipes remain. The details of selected categories and recipes are shown in Table 2, in which the *Voice assistants* represents categories for channels such as Amazon Alexa and Google Assistant. In addition, each recipe consists of a trigger and an action that may belong to different categories. For example, Fig. 6 shows the recipe R_4 has the trigger T_2 and the action A_3 , which belong to different channels C_1 , C_2 . If C_1 and C_2 belong to different categories, R_4 has two categories. Therefore, each recipe may belong to two categories, which is why the total number of recipes in the last column in Table 2 is greater than 2173.

For node classification, we use the filtered data to build IoT-HIN according to Section 4.2. IoT-HIN contains 4 types of nodes, 5 types of edges, and 12 types of class labels, which the statistics of IoT-HIN are illustrated in Table 3. The number of nodes trigger, action, word, and recipe are 202, 460, 2656, and 2173, respectively. The category of recipes is seen as the class label and can be predicted through the multi-label classification.

5.1.2. Compared methods

To validate the efficiency of our framework, we select the following classical NLP and HIN embedding algorithms as comparison methods.

TF-IDF (Ramos et al., 2003): It is a classical statistical method that can evaluate the importance of a word or a sentence to an article or a corpus in the form of vector weight. We can consider the vector weight as embedding for each description.

sentence-bert (Reimers & Gurevych, 2019): It can generate sentence embeddings for sentences in text, and the semantic similarity of the sentence can be calculated. We apply it to recipes' descriptions to generate embeddings.

node2vec (Grover & Leskovec, 2016): It is a typical network embedding method for homogeneous networks. It designs a biased random walk strategy to sample the neighborhood nodes, which can smoothly adjust between breadth-first sampling and depth-first sampling using two hyperparameters p and q . The sampled paths can be used to generate node embeddings. To apply node2vec to heterogeneous graphs, we treat the nodes in IoT-HIN as the same type.

metapath2vec (Dong et al., 2017): It is a heterogeneous network embedding algorithm. It applies a rather strict single metapath scheme to guide random walks, using negative sampling for specific types of context.

metagraph2vec (Zhang et al., 2018): It uses a metagraph-guided random walk to generate heterogeneous node sequences. The skip-gram technique is used to learn node embedding. Different from metapath2vec, the metagraph2vec can be applied for multiple metapath schemes.

FastGTNs (Yun et al., 2022): It is a supervised learning method and combines the advantage of transformer and graph neural networks (Yun, Jeong, Kim, Kang, & Kim, 2019) to extract useful knowledge from heterogeneous graphs and learn better representation.

5.1.3. Evaluation metrics

Due to class imbalance, the above-mentioned algorithms classify nodes into the majority class, which tends to perform better. In our work, we analyze the performance of TAPFuser using the Micro-F1 and Macro-F1 metrics, which are widely used to evaluate multi-label classification (Dong et al., 2017; Zhang et al., 2020, 2018).

Suppose S is the whole label set. Given a label s , $TP(s)$ denotes the number of correct predictions we call true positives (TP). $FP(s)$ and $FN(s)$ indicate the number of false positives (FP) and false negatives (FN) separately. Moreover, FP means the prediction is positive, actually negative; FN means the prediction is negative, actually positive. Micro-F1 calculates F1 globally by counting the total TP, FP, and FN, giving equal weight to each instance. Macro-F1 is the average F1 over the overall set of labels S . In detail, they are defined as follows:

$$Precision(S) = \frac{\sum_{s \in S} TP(s)}{\sum_{s \in S} (TP(s) + FP(s))} \quad (5)$$

$$Recall(S) = \frac{\sum_{s \in S} TP(s)}{\sum_{s \in S} (TP(s) + FN(s))} \quad (6)$$

$$Micro-F1(S) = \frac{2 \cdot Precision(S) \cdot Recall(S)}{Precision(S) + Recall(S)} \quad (7)$$

$$Macro-F1(S) = \frac{\sum_{s \in S} F1(s)}{|S|} \quad (8)$$

where $|S|$ denotes the number of labels in set S .

5.1.4. Parameter setting

For a fair comparison, the embedding dimension is set to 200 for all the above methods. For all random walk-based methods, we conduct 40 random walks per node with a walk length of 15, and the neighborhood size is set to 5 with 5 negative samples. For node2vec, we set p and q as 1. For FastGTNs, we use the default parameters used in the literature (Yun et al., 2022) and do not tune hyperparameters. For our framework TAPFuser, we use the same parameters

Table 4

The used metapaths and metagraphs.

Network type	Metapath	Metagraph
ICN	$RAR(P_1)$, $RTR(P_2)$	$R(T/A)R(M_1)$
HDDN	$RWR(P_3)$	
IoT-HIN	–	$R(T/A/W)R(M_2)$

as random walk-based methods. On IoT-HIN, we use tenfold cross-validation with 20% labeled data to perform a grid search over $\lambda \in \{0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1\}$ and obtain the hyperparameter $\lambda = 0.8$. On ICN, we use the same method as IoT-HIN to learn the values of hyperparameter λ .

Besides, metapath and metagraph-based methods must specify predefined metapaths and metagraphs. Following (Sun et al., 2011; Zhang et al., 2018), we only select short metapaths since long metapaths are likely to introduce noise. Metapaths and metagraphs we used are shown in Table 4. P_1 , P_2 and P_3 are designed by the metapath scheme, which is defined in metapath2vec (Dong et al., 2017) for different network type. M_1 and M_2 are designed by metagraph2vec and our TAPFuser for different network type. The detailed illustrations are shown in Fig. 8.

5.2. Multi-label classification

In this section, the multi-label classifications are carried out on the learned embeddings of recipes to compare the performance of different algorithms. The learned embeddings are used to train one-vs-rest linear classifiers using the LIBLINEAR package (Fan, Chang, Hsieh, Wang, & Lin, 2008). We randomly split the dataset into training and testing sets, and the training ratio varies from 10% to 90%. The experiments are conducted on the three networks we constructed and evaluated the classification accuracy with Micro-F1 and Macro-F1 scores. The results are reported in Table 5. The best result is highlighted in bold.

From Table 5, there are three significant findings we summarize as follows:

1. TAPFuser (M_1) is better than other compared methods on the ICN network in terms of the Micro-F1 and Macro-F1 score when the training ratio is 10%~90%.
2. For HDDN built by human description from the human level, TAPFuser (P_3) with just 10% labeled nodes outperforms other compared methods in terms of Micro-F1, such as TF-IDF, sentence-bert, node2vec, and metapath2vec (P_3).
3. TAPFuser (M_2) works very well in IoT-HIN, achieving an average of 2%, and 2% and 13% improvements in Macro-F1 score over node2vec, metagraph2vec (M_2) and FastGTNs, respectively, when the training ratio is 10%~90%. Meanwhile, TAPFuser (M_2) in IoT-HIN outperforms all other compared methods in ICN and HDDN and gains of 8% Micro-F1 score on average over the state-of-the-art methods.

5.3. Time complexity analysis

Our framework mainly contains two procedures: (1) generating random walk sequences and (2) training. For the random walk phase in an IoT-HIN $G = (V, E, T)$ with walks per node h and walk length g , the walk complexity is $O(h \cdot g \cdot |T_V| \cdot |V|)$, where $|T_V|$ is the number of node types. For the training phase, the skip-gram is used to train the embedding of nodes. For each node pair, the training time complexity is $O(K \cdot d)$, where K is the number of negative samples, and d is the dimension of the node. The total time complexity is $O(h \cdot g \cdot |T_V| \cdot |V| \cdot c \cdot K \cdot d)$. Since h, g, c, K , and d are constants much smaller than $|V|$, the main complexity is $O(|T_V| \cdot |V| \cdot d)$, which is linear to the number of nodes.

By contrast, the time complexity of TF-IDF is $O(m \cdot n)$, where n is the number of recipes, and m is the number of words. The time

Table 5
Multi-label Classification Results for Recipes on IFTTT Dataset.

Metric	Network	Method	Portion of Data Set Used for Training								
			10%	20%	30%	40%	50%	60%	70%	80%	90%
Micro-F1	ICN	node2vec	0.6667	0.6832	0.7157	0.7257	0.7371	0.7393	0.7481	0.7393	0.7662
		metapath2vec (P1)	0.4044	0.4485	0.4805	0.4778	0.4804	0.4807	0.4961	0.4948	0.4954
		metapath2vec (P2)	0.6177	0.6194	0.6388	0.6453	0.6531	0.6518	0.6577	0.6512	0.6458
		metagraph2vec (M1)	0.7164	0.7589	0.7783	0.7888	0.8039	0.8101	0.8253	0.8262	0.838
		TAPFuser (M1)	0.7384	0.761	0.7893	0.8004	0.8081	0.8182	0.8292	0.832	0.8519
	HDDN	TF-IDF	0.6368	0.7108	0.7392	0.7563	0.7742	0.791	0.7991	0.8158	0.8356
		sentence-bert	0.657	0.7199	0.7466	0.7562	0.7584	0.771	0.7757	0.7882	0.7875
		node2vec	0.6274	0.6595	0.6794	0.6952	0.6902	0.6989	0.6944	0.6912	0.707
		metapath2vec (P3)	0.6331	0.6574	0.6838	0.6905	0.6977	0.7042	0.7092	0.7053	0.7047
		TAPFuser (P3)	0.6499	0.6728	0.6935	0.6967	0.7032	0.7034	0.7141	0.7184	0.75
	IoT-HIN	node2vec (IoT-HIN)	0.7716	0.8059	0.832	0.8383	0.841	0.8455	0.8516	0.8656	0.8773
		metagraph2vec (M2)	0.7956	0.8213	0.839	0.8468	0.8503	0.8605	0.874	0.8714	0.8634
		FastGTNs	0.8259	0.8386	0.8447	0.8481	0.8539	0.8557	0.8578	0.8594	0.8594
		TAPFuser (M2)	0.7969	0.8291	0.8506	0.8627	0.8744	0.8832	0.8833	0.8841	0.9005
Macro-F1	ICN	node2vec	0.608	0.6396	0.6741	0.6878	0.6989	0.704	0.7104	0.7029	0.7517
		metapath2vec (P1)	0.3441	0.4067	0.4296	0.4292	0.4371	0.4332	0.4497	0.4465	0.461
		metapath2vec (P2)	0.5741	0.5765	0.6021	0.6108	0.6168	0.6088	0.6074	0.6083	0.6179
		metagraph2Vec (M1)	0.665	0.7275	0.7545	0.7765	0.7879	0.7901	0.8038	0.8109	0.828
		TAPFuser (M1)	0.6914	0.7332	0.7699	0.7836	0.7985	0.8036	0.8112	0.8202	0.8376
	HDDN	TF-IDF	0.5067	0.6205	0.6669	0.6907	0.7202	0.7429	0.7437	0.7664	0.7985
		sentence-bert	0.5428	0.6277	0.6649	0.6819	0.6935	0.7187	0.7342	0.7342	0.7352
		node2vec	0.5625	0.6051	0.6343	0.6513	0.6466	0.6615	0.6557	0.6612	0.6829
		metapath2vec (P3)	0.5584	0.5964	0.6264	0.6444	0.6597	0.6622	0.6693	0.6712	0.6977
		TAPFuser (P3)	0.583	0.6233	0.6406	0.65	0.6601	0.6662	0.6768	0.6771	0.7016
	IoT-HIN	node2vec (IoT-HIN)	0.7324	0.7754	0.8087	0.8239	0.8259	0.8334	0.8371	0.8447	0.859
		metagraph2vec (M2)	0.7489	0.7899	0.815	0.8261	0.8306	0.842	0.855	0.8423	0.832
		FastGTNs	0.6765	0.7036	0.7194	0.728	0.741	0.7439	0.7447	0.7463	0.7456
		TAPFuser (M2)	0.7525	0.7935	0.8278	0.8474	0.8592	0.867	0.8642	0.8668	0.8942

complexity of sentence-bert is $O(n^2 \cdot d)$. Node2vec does not need to consider the node type, and its time complexity is $O(|V| \cdot d)$. The time complexity of metapath2vec and metagraph2vec is the same as our time complexity. The time complexity of FastGTNs requires at least $O(|V|^3)$ due to complex matrix multiplication operations. Based on the above computational complexity analysis, our framework TAPFuser has relatively high computational efficiency.

5.4. Parameter sensitivity

In this subsection, we analyze our proposed model by studying the impact of the hyperparameter λ , dimensions, and walk length. The multi-label classification is evaluated the performance of the TAPFuser.

5.4.1. Impact of λ

The hyperparameter λ is used to adjust the walking probability of TAPFuser between the human and machine levels. The classification task is evaluated under training on 20% labeled nodes, and the results are shown in Fig. 9(a).

The specific walking probability $\lambda = 0$ and $\lambda = 1$ indicate that TAPFuser only learns on HDDN and ICN, respectively. We observe that TAPFuser performs poorly on both networks, especially on HDDN, implying that a single level of data leads to a loss of semantic information for IoT. We can also see that TAPFuser performs best when λ reaches 0.8. This observation means that the information from the machine level makes TAPFuser more profitable for classification tasks than the human level. Meanwhile, it demonstrates the need to integrate information from the human and machine levels through the graph model.

We also provide a detailed analysis regarding the property of IoT-HIN in terms of degree distribution. As shown in Fig. 10(a), the degree distribution of ICN is scattered. We can find that the distribution of IoT-HIN is smooth, which is beneficial for the sampling process. Furthermore, distributions of node types in IoT-HIN are analyzed in Fig. 10(b). The recipe, trigger, and action nodes are well connected and contain numerous star-shaped metagraphs. While the word nodes

follow the power-law degree distribution, providing abundant links for IoT-HIN integration.

5.4.2. Impact of embedding dimensions

We further explore the impact of the embedding dimension d on the performance of our TAPFuser. We vary the dimension between 16 and 500 and then perform TAPFuser with 10% labeled nodes on the classification task. In Fig. 9(b), we observe that the performance of the TAPFuser is not the best when d is too small and large. The reason is that small dimensions cannot preserve all potentially useful information well, and large dimensions often lead to over-parameterization. In our experiment, the best value of d is 200.

5.4.3. Impact of walk length

Finally, we investigate the impact of walk length on the multi-label classification and running time during model training. The result is shown in Fig. 9(c). We observe that the performance and running time of the TAPFuser rise as the walk length increases. While the performance of the TAPFuser descends when the walk length is set to 20. In our experiment, we set the walk length to 15, which is the trade-off between the efficiency and performance of our model.

5.5. Discussion

In this section, we investigate the results observed in Section 5.2 and obtain some insights.

1. From the first result, we observe that metagraph-based methods, TAPFuser (M_1) and metagraph2vec (M_1), outperform metapath-based metapath2vec (P_1) and metapath2vec (P_2). The main reason for achievement is that the personalized metagraph M_1 we designed can characterize multiple relationships generated by entity interaction from the machine level. Although metagraph2vec can achieve this goal, TAPFuser can capture more semantic information through specific sampling methods. Meanwhile, by considering multiple metapaths between different

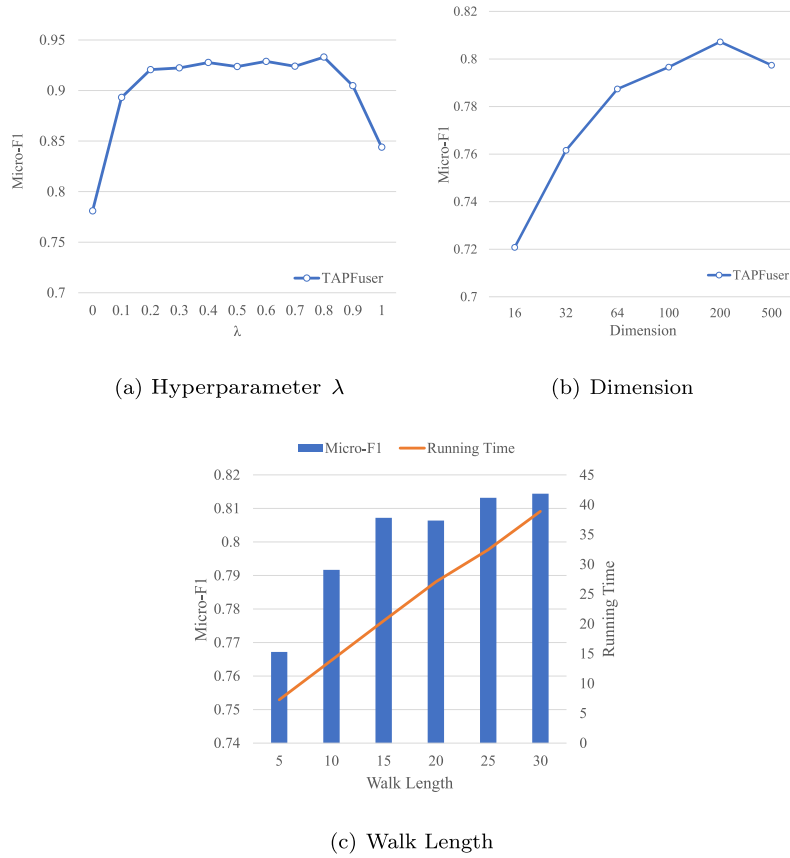


Fig. 9. Impact of hyperparameter λ , dimension and walk length on node classification.

- types of nodes, TAPFuser can learn better embeddings than node2vec, which neglects different node types and edge types.
- According to the second result, the graph-based methods, including TAPFuser (P_3), metapath2vec (P_3), and node2vec, are superior to the NLP-based methods, i.e., TF-IDF and sentencebert, in HDDN when labeled data is 10%. As the labeled data increases during training, the performance of NLP-based methods is significantly better than that of graph-based methods. That means modeling human demands through the graph is reasonable and practical since labeled data is expensive in real-world IoT scenarios.
 - According to the second result, when labeled data is only 10%, graph-based methods, including TAPFuser (P_3) and node2vec, outperform NLP-based methods on HDDN such as TF-IDF and sentencebert. However, as the amount of labeled data increases, the performance of NLP-based methods becomes significantly better than that of graph-based methods. This suggests that modeling human demands through graphs is reasonable and practical, especially considering the expensive nature of labeled data in real-world IoT scenarios.
 - The third result shows the favorable performance of TAPFuser (M_2), demonstrating the validity of formulating the mapping problem as a multi-label classification problem. Notably, we observe that TAPFuser (M_2) outperforms TAPFuser (M_1) and TAPFuser (P_3), providing evidence for the effectiveness of integrating information from both machine and human levels within our framework. It is worth mentioning that when the labeled data accounts for 10% and 20%, FastGTNs outperforms TAPFuser in terms of Micro-F1 score due to its enhanced ability to capture the graph structure. It is important to note that the learning process of FastGTNs is supervised and necessitates labeled data for model training. In contrast, TAPFuser operates

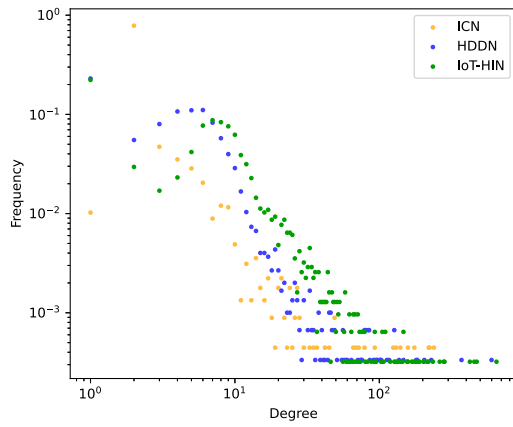
in an unsupervised manner, eliminating the need for node labels during model training.

Our proposed framework can learn much better heterogeneous node embeddings by integrating data from different levels. Multi-label classification experiments on a real-world dataset have proved it. The learned embeddings will be applied in information retrieval, service recommendation, and clustering (Shen & Chung, 2020).

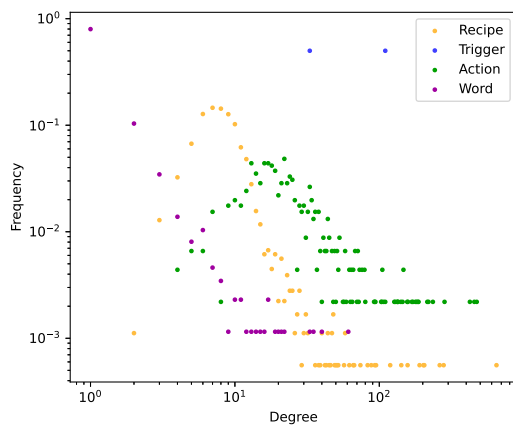
- A user can retrieve recipes with high similarity in topics or service categories in IFTTT. For example, given a category *Email*, we can search out the recipe for “Get Gmail notifications for new email” and the recipe for “Send RSS (Really Simple Syndication) to email”, which belong to *Email* service category.
- Recommendation of recipes according to user’s preferences since semantically similar recipes are close to each other in vector space. A user who has created a recipe for “Get Amazon Free App of the day information via email! Daily!” will be recommended the recipes “Email me the free App of the day” and “Amazon ‘deal of the day’ daily email #amazon.”
- In entity resolution applications, duplicate IoT-HIN entity pairs (e.g., two different entities in IoT-HIN referring to the same recipe), which have high similarity, can be identified and deleted based on their embeddings to achieve the purpose of multi-source data cleaning and reuse in IoT.

6. Conclusion

With the expansion of connected entities and functions in IoT, it has become increasingly impractical for humans to customize recipes to meet their specific demands manually. To address this challenge, this article introduces TAPFuser, an IoT data fusion framework that



(a) ICN, HDDN and IoT-HIN



(b) IoT-HIN

Fig. 10. Degree distribution of different network and node types in IoT-HIN.

aims to establish a mapping between human demands and recipes, enabling IoT automation. TAPFuser divides the TAP data into machine and human levels based on their sources. The data from these levels are then modeled as an IoT-HIN, leveraging the graph to capture the relationships and interactions among entities. The framework employs HIN embedding techniques to learn node vector representations, enabling structural information utilization within the IoT-HIN. To overcome the challenges posed by network heterogeneity and semantic diversity, TAPFuser introduces a specific metagraph-guided random walk method. This method captures level-aware semantics within the IoT-HIN, effectively addressing the complexities of the network. Finally, the mapping between human demands and recipes is formulated as a multi-label classification problem on the IoT-HIN. Extensive experiments conducted on a real-world IoT dataset demonstrate the effectiveness of the proposed TAPFuser framework.

CRedit authorship contribution statement

Gang Wu: Conceptualization, Methodology, Validation, Visualization, Writing – original draft. **Liang Hu:** Supervision, Resources. **Xuelin Mao:** Software, Writing – review & editing. **Yongheng Xing:** Investigation. **Feng Wang:** Investigation, Methodology, Conceptualization, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This work is funded by the Science and Technology Development Plan of Jilin Province of China under Grant 20220101115JC, and by National Key R&D Plan of China under Grant No. 2017YFA0604500, and by Project of Jilin Province Development and Reform Commission, China No. 2019FGWTZC001.

References

- Cai, H., Zheng, V. W., & Chang, K. C. C. (2018). A comprehensive survey of graph embedding: Problems, techniques, and applications. *IEEE Transactions on Knowledge and Data Engineering*, 30(9), 1616–1637.
- Corno, F., De Russis, L., & Monge Roffarello, A. (2019). RecRules: Recommending IF-THEN rules for end-user development. *ACM Transactions on Intelligent Systems and Technology*, 10(5).
- Corno, F., De Russis, L., & Monge Roffarello, A. (2021). From users' intentions to if-then rules in the internet of things. *ACM Transactions on Information Systems (TOIS)*, 39(4), 1–33.
- Dong, Y., Chawla, N. V., & Swami, A. (2017). Metapath2vec: Scalable representation learning for heterogeneous networks. In *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 135–144). ACM.
- Fan, R. E., Chang, K. W., Hsieh, C. J., Wang, X. R., & Lin, C. J. (2008). LIBLINEAR: A library for large linear classification. *The Journal of Machine Learning Research*, 9, 1871–1874.
- Grover, A., & Leskovec, J. (2016). Node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 855–864). ACM.
- He, Y., Song, Y., Li, J., Ji, C., Peng, J., & Peng, H. (2019). Heterospacewalk: A heterogeneous spacey random walk for heterogeneous information network embedding. In *Proceedings of the 28th ACM international conference on information and knowledge management* (pp. 639–648).
- Hu, L., Gong, Y., Xing, Y., & Wang, F. (2019a). Semantic representation with heterogeneous information network using matrix factorization for clustering in the internet of things. *IEEE Access*, 7, 31233–31242.
- Hu, H., Tang, B., Zhang, Y., & Wang, W. (2020). Vehicular ad hoc network representation learning for recommendations in internet of things. *IEEE Transactions on Industrial Informatics*, 16(4), 2583–2591.
- Hu, L., Wu, G., Xing, Y., & Wang, F. (2019b). Things2Vec: Semantic modeling in the internet of things with graph representation learning. *IEEE Internet of Things Journal*, 7(3), 1939–1948.
- Hu, L., Xing, Y., Gong, Y., Zhao, K., & Wang, F. (2019c). Nonnegative matrix tri-factorization with user similarity for clustering in point-of-interest. *Neurocomputing*, 363, 58–65.
- Kim, Y., An, J., Lee, M., & Lee, Y. (2017). An activity-embedding approach for next-activity prediction in a multi-user smart space. In *2017 IEEE international conference on smart computing* (pp. 1–6). IEEE.
- Kim, Y., Jung, S., Ji, S., Hwang, E., & Rho, S. (2019). IoT-based personalized NIE content recommendation system. *Multimedia Tools and Applications*, 78(3), 3009–3043.
- Kim, S., Suh, Y., & Lee, H. (2022). What IoT devices and applications should be connected? Predicting user behaviors of IoT services with node2vec embedding. *Information Processing & Management*, 59(2), Article 102869.
- Mi, X., Qian, F., Zhang, Y., & Wang, X. (2017). An empirical characterization of IFTTT: ecosystem, usage, and performance. In *Proceedings of the 2017 internet measurement conference* (pp. 398–404). ACM.
- Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013a). Efficient estimation of word representations in vector space. arXiv preprint arXiv:1301.3781.
- Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013b). Distributed representations of words and phrases and their compositionality. In *Advances in neural information processing systems* (pp. 3111–3119).
- Ramos, J., et al. (2003). Using tf-idf to determine word relevance in document queries. In *Proceedings of the first instructional conference on machine learning*, Vol. 242 (pp. 133–142). Piscataway, NJ.

- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using siamese BERT-networks. In K. Inui, J. Jiang, V. Ng, & X. Wan (Eds.), *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing, EMNLP-IJCNLP 2019, Hong Kong, China, November 3-7, 2019* (pp. 3980–3990). Association for Computational Linguistics, <http://dx.doi.org/10.18653/v1/D19-1410>.
- Shen, X., & Chung, F.-L. (2020). Deep network embedding for graph representation learning in signed networks. *IEEE Transactions on Cybernetics*, 50(4), 1556–1568.
- Sun, Y., Han, J., Yan, X., Yu, P. S., & Wu, T. (2011). Pathsim: Meta path-based top-k similarity search in heterogeneous information networks. *Proceedings of the VLDB Endowment*, 4(11), 992–1003.
- Ur, B., McManus, E., Pak Yong Ho, M., & Littman, M. L. (2014). Practical trigger-action programming in the smart home. In *Proceedings of the SIGCHI conference on human factors in computing systems* (pp. 803–812).
- Ur, B., Pak Yong Ho, M., Brawner, S., Lee, J., Mennicken, S., Picard, N., et al. (2016). Trigger-action programming in the wild: An analysis of 200,000 ifttt recipes. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems* (pp. 3227–3231).
- Wang, F., Hu, L., Sun, R., Hu, J., & Zhao, K. (2018). SRMCS: A semantic-aware recommendation framework for mobile crowd sensing. *Information Sciences*, 433.
- Wei, D., Ning, H., Shi, F., Wan, Y., Xu, J., Yang, S., et al. (2021). Dataflow management in the internet of things: Sensing, control, and security. *Tsinghua Science and Technology*, 26(6), 918–930.
- Xing, Y., Hu, L., Zhang, X., Wu, G., & Wang, F. (2021). Nonnegative matrix factorization based heterogeneous graph embedding method for trigger-action programming in IoT. *IEEE Transactions on Industrial Informatics*, 18(2), 1231–1239.
- Yao, Z., Li, X., Gao, J., Sadler, B., & Sun, H. (2019). Interactive semantic parsing for if-then recipes via hierarchical reinforcement learning. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 33 (01), (pp. 2547–2554).
- Yu, H., Hua, J., & Julien, C. (2021). Analysis of ifttt recipes to study how humans use internet-of-things (iot) devices. In *Proceedings of the 19th ACM conference on embedded networked sensor systems* (pp. 537–541).
- Yun, S., Jeong, M., Kim, R., Kang, J., & Kim, H. J. (2019). Graph transformer networks. *Advances in Neural Information Processing Systems*, 32.
- Yun, S., Jeong, M., Yoo, S., Lee, S., Sean, S. Y., Kim, R., et al. (2022). Graph transformer networks: Learning meta-path graphs to improve GNNs. *Neural Networks*, 153, 104–119.
- Yusuf, I. N. B., Jamal, D. B. A., Jiang, L., & Lo, D. (2022a). RecipeGen++: An automated trigger action programs generator. In *Proceedings of the 30th ACM joint European software engineering conference and symposium on the foundations of software engineering* (pp. 1672–1676).
- Yusuf, I. N. B., Jiang, L., & Lo, D. (2022b). Accurate generation of trigger-action programs with domain-adapted sequence-to-sequence learning. In *Proceedings of the 30th IEEE/ACM international conference on program comprehension* (pp. 99–110).
- Zangeneh, P., & McCabe, B. (2020). Ontology-based knowledge representation for industrial megaprojects analytics using linked data and the semantic web. *Advanced Engineering Informatics*, 46, Article 101164.
- Zhang, W., Fang, Y., Liu, Z., Wu, M., & Zhang, X. (2020). mg2vec: Learning relationship-preserving heterogeneous graph representations via metagraph embedding. *IEEE Transactions on Knowledge and Data Engineering*, 34(3), 1317–1329.
- Zhang, D., Yin, J., Zhu, X., & Zhang, C. (2018). Metagraph2vec: Complex semantic path augmented heterogeneous network embedding. In *Advances in knowledge discovery and data mining: 22nd Pacific-Asia conference, PAKDD 2018, Melbourne, VIC, Australia, June 3-6, 2018, Proceedings, Part II* 22 (pp. 196–208). Springer.